

Discovering Persistent Behavioural Patterns for Interpretable Blockchain Forensics

Dorottya Zelenyanszki^{a,*}, Zhé Hóu^a, Kamanashis Biswas^b and Vallipuram Muthukkumarasamy^a

^aGriffith University, 170 Kessels Road, Nathan, 4111, QLD, Australia

^bAustralian Catholic University, 1100 Nudgee Road, Banyo, 4014, QLD, Australia

ARTICLE INFO

Keywords:
blockchain
DeFi
sentence embedding
sequence model
clustering
user profiling

ABSTRACT

Public blockchain data enables large-scale DeFi-related analysis, but many existing approaches are application-specific, difficult to scale, or hard to interpret. This research proposes a scalable, application-agnostic framework for *persistent behavioural pattern discovery* from large-scale blockchain activity. It constructs behaviour sentences enriched with contract, token and market context, then applies a two-step embedding process: sentence-level embeddings capture individual actions, while sequence-level embeddings capture user behaviour over time. An interpretable behavioural profiler characterizes discovered communities through behavioural motifs, routines, temporal dynamics, entity exposure, and suspiciousness evidence. Evaluation on Ethereum using over 30 million transactions shows that the framework uncovers both routine and malicious behavioural patterns, including decentralised exchange (DEX) trading, NFT activity, phishing, bot operations, oracle manipulation, and rug-pull schemes. Importantly, many patterns remain stable across independent observation windows, enabling the identification of long-term behaviours beyond a single analysis period. The proposed framework combines scalability, interpretability, and persistence analysis, supporting blockchain forensic investigation, behavioural attribution, and threat discovery.

1. Introduction

DeFi has rapidly evolved into a large ecosystem of smart contracts, tokens, liquidity pools, and user-driven protocol interactions. Its security challenges extend beyond smart-contract correctness to protocol design, market structure, and the economic incentives governing participant behaviour (Werner et al., 2023). At the implementation level, mechanisms such as upgradeable smart contracts can introduce framework-specific faults and complicate the testing of upgrade paths, potentially resulting in deployed vulnerabilities and security incidents (Zhong et al., 2025b). The operational consequences remain substantial. For example, the De.Fi REKT report for October 2025¹ documented losses of \$38.63 million across nine exploits in a single month. Detecting and understanding such risks requires analysing blockchain data that is publicly accessible but heterogeneous, massive in volume, and inherently temporal, making it both a challenging and promising setting for machine learning (Azad et al., 2025a). Public availability, however, does not make on-chain behaviour immediately interpretable. Transactions, logs, contract calls, token movements, and event arguments provide low-level records of activity, but do not directly reveal user roles, recurring routines, or persistent behavioural patterns. A central challenge is therefore to transform large volumes of heterogeneous on-chain activity into representations that support population-level behavioural analysis, rather than limiting analysis to individual transactions or incidents.

Existing work has advanced blockchain security and DeFi risk analysis in several directions. These include forensic tracing and suspicious-flow reconstruction (Lin et al., 2025a,b), behavioural representation learning and user classification (Azad et al., 2025b; Bonifazi et al., 2022), and attack- or scam-specific detection (Xu et al., 2025; Wang et al., 2024a; Zhong et al., 2025a; Torres et al., 2019). These approaches are valuable, and, in several cases, highly scalable within their own problem settings, but they are often transaction- or flow-centric, attack-specific, label-dependent, tied to particular protocols, or focused on individual suspicious entities. They therefore provide

*Corresponding author

✉ dora.zelenyanszki@griffithuni.edu.au (D. Zelenyanszki); z.hou@griffith.edu.au (Z. Hóu); Kamanashis.Biswas@acu.edu.au (K. Biswas); v.muthu@griffith.edu.au (V. Muthukkumarasamy)

ORCID(s): 0009-0004-1300-0357 (D. Zelenyanszki); 0000-0001-7164-0580 (Z. Hóu); 0000-0003-3719-8607 (K. Biswas); 0000-0002-6787-6379 (V. Muthukkumarasamy)

¹<https://tinyurl.com/t2e294nz>

limited support for discovering broad, persistent, and interpretable behavioural structure across large user populations, especially when suspicious evidence is sparse, mixed with ordinary activity, and evolves over time. In particular, they do not directly address deriving long-term, persistent behavioural patterns from millions of transactions while jointly incorporating direct suspicious labels and exposure-based suspicious evidence. They also provide limited support for explaining how users organise into recurring behavioural groups, how suspicious evidence is embedded within ordinary activity, and whether these behavioural patterns persist across independent and longer observation windows.

This paper addresses these gaps through a scalable, application-agnostic framework for *persistent behavioural pattern discovery* of Ethereum users. It transforms raw transaction activity into behavioural sentences, embeds them through a two-step representation process, clusters users according to their sequence-level behaviour, and profiles the resulting groups using behavioural and suspiciousness-oriented evidence. The framework operates on user histories, preserves sequential activity, and does not require labels for malicious or benign, but to reveal how ordinary and suspicious behaviours are organised across the population and whether these patterns persist over time. It also produces interpretable cluster-level and tag-level profiles while remaining practical at million-user scale.

The main contributions of this paper are threefold:

- We propose a scalable, application-agnostic framework for *persistent behavioural pattern discovery* that converts blockchain transactions and decoded logs into behavioural sentences preserving action order, event semantics, entity types, and market signals learns user-level sequence representations, clusters users without labels, and profiles the resulting behavioural groups.
- We establish the most suitable framework setting through a systematic, multi-stage evaluation of sentence embedding models, sequence aggregation methods, training objectives, sequence-length strategies, clustering algorithms and resolutions, and behavioural semantics, balancing representation quality, clustering structure and stability, scalability, length leakage, and the concentration of pre-labelled behaviours.
- We develop an interpretable behavioural profiler that explains discovered clusters through behavioural motifs, ordered routines, activity and temporal patterns, diversity, concentration, entity exposure, and separated direct-label and exposure-based suspiciousness evidence, enabling the identification of persistent ordinary and suspicious behavioural patterns across independent monthly and cumulative multi-month windows.

The remainder of the paper is organised as follows. Section 2 reviews the relevant literature, and Section 3 presents the proposed framework. Section 4 describes the experiments. Section 5 examines the resulting behavioural profiles and their persistence across observation windows. Sections 6 and 7 discuss the limitations and conclude the paper.

2. Related work

Relevant research on blockchain security and DeFi risk includes forensic tracing and analysis, behavioural representation learning, attack-specific detection, pre-incident attack inference, and governance- or ecosystem-level analysis. Although these directions have improved visibility into unsafe activity, they often focus on specific attacks, contracts, transactions, labelled classification tasks, or curated cases, limiting their ability to recover unsupervised, interpretable, and long-term behavioural patterns across large user populations.

Forensic tracing systems focus on reconstructing suspicious transaction flows. Lin et al. (2025a) propose RiskTagger, an LLM-guided agent that extracts incident clues, traces laundering-related fund flows through iterative reasoning over on-chain evidence, and generates evidence-grounded forensic reports. Lin et al. (2025b) present ABCTRACER, which combines semantic log extraction, named entity recognition, and information retrieval to trace suspicious cross-chain bridge flows. These systems demonstrate the value of semantic recovery and reasoning for identifying and explaining suspicious flows, but do not aim to recover stable behavioural organisation across large user populations.

Behavioural representation learning is more closely related to our setting. Azad et al. (2025b) introduce Chainlet Orbits, which captures structural roles in temporal blockchain graphs to produce scalable and interpretable address embeddings. Bonifazi et al. (2022) represent Ethereum users through multivariate temporal profiles for multi-class classification. Tovanich and Cazabet (2023) learn Bitcoin taint-flow fingerprints by extracting taint-flow graphs, sampling random walks, and embedding money-flow patterns for classification and clustering. These studies demonstrate that blockchain entities can be represented through structural, temporal, and flow-based behaviour, but they mainly support address classification, Bitcoin money-flow analysis, or clustering without profiler-based interpretation. In contrast, our framework models Ethereum user histories using smart-contract calls, decoded events, token standards, approvals,

Table 1

Positioning of the related work. *Hist.*, *Seq.*, *Unsup.*, *Agn.*, *Prof.*, and *LT* denote user-history modelling, sequence-aware modelling, unsupervised discovery, application-agnostic analysis, interpretable profiling, and long-term persistence, respectively. Filled, half-filled, and unfilled circles denote yes, partial, and no.

Related work	Hist.	Seq.	Unsup.	Agn.	Prof.	LT
Forensic tracing and analysis	●	●	○	●	●	○
Bitcoin taint-flow fingerprints	●	●	●	●	○	○
Representation and classification	●	●	○	●	●	●
Vulnerability and attack detection	●	●	○	○	●	○
Governance and ecosystem studies	●	●	●	○	●	●
Proposed work	●	●	●	●	●	●

swaps, NFT movements, and protocol interactions, then interprets the resulting clusters using post hoc suspiciousness evidence and cross-window persistence analysis.

Vulnerability and attack-specific detection methods are effective within predefined threat classes. Xu et al. (2025) formalise attack behaviour using temporal logic for runtime monitoring in MoE. Wang et al. (2024a) propose DeFiGuard, which detects price manipulation from transaction-derived cash-flow graphs, whereas Zhong et al. (2025a) use large language models to recover DeFi operations and detect price manipulation. Ibba et al. (2025) combine software metrics and topic modelling with machine-learning classifiers to detect vulnerabilities in Ethereum smart contracts, reporting 97.7% accuracy for binary classification and an improved F1-score of 0.881 after incorporating topic information. Wu et al. (2025) use temporal graph neural networks to detect rug-pull risks, with memory and attention mechanisms used to model dynamic token-transaction patterns, while Torres et al. (2019) use symbolic execution to detect honeypot contracts. These methods are effective for detecting known vulnerabilities, attacks, and scam mechanisms within well-defined settings, but their reliance on specialised threat definitions, labelled data, or contract-level analysis makes them less suitable for broad behavioural discovery across large user populations, particularly when the aim is to identify latent suspicious groups without predefining the behaviour of interest.

Pre-incident inference and ecosystem-level studies address other dimensions of blockchain risk. Wang et al. (2024b) analyse adversarial contracts before exploit transactions to infer likely attack intentions, but their approach remains contract- and incident-centric. Although this supports operational defence before an exploit occurs, it does not target stable user-level behavioural organisation. Broader studies examine DAO governance weaknesses (Ma et al., 2025), rug-pull causes, datasets, and detection tools (Sun et al., 2024), and longitudinal token and liquidity-pool activity involving token spammers, disposable-token rug pulls, and sniper bots (Cernera et al., 2023). The governance analysis shows that risk can arise from both technical and procedural failures, while the rug-pull study highlights the limited coverage of existing datasets and tools. The longitudinal ecosystem analysis further identifies token spammers, disposable-token rug pulls, and sniper bots as recurring malicious roles. These studies broaden blockchain risk analysis by examining governance processes, rug-pull causes and dataset coverage, and longitudinal token and liquidity-pool ecosystems. However, each addresses a particular aspect of the ecosystem rather than providing a general framework for discovering and interpreting behavioural patterns across large user populations.

Overall, existing work provides strong foundations in forensic tracing, behavioural representation, vulnerability and attack detection, and ecosystem analysis. However, these lines of work do not jointly support large-scale user-history representation, sequence-aware modelling, unsupervised grouping, post hoc interpretation using suspiciousness evidence, and interpretable behavioural profiling. Our framework addresses this gap by combining sentence-based behavioural abstraction, sequence-level embedding, clustering, and profiling. Table 1 summarises this positioning.

3. The proposed framework

This section presents the proposed framework. It abstracts transactions and logs into behavioural sentences, embeds them into user-level representations, and clusters users without labels. The resulting groups are then profiled using shared routines, entity exposure, temporal patterns, and post hoc suspiciousness evidence. The framework is visualised in Fig. 1.

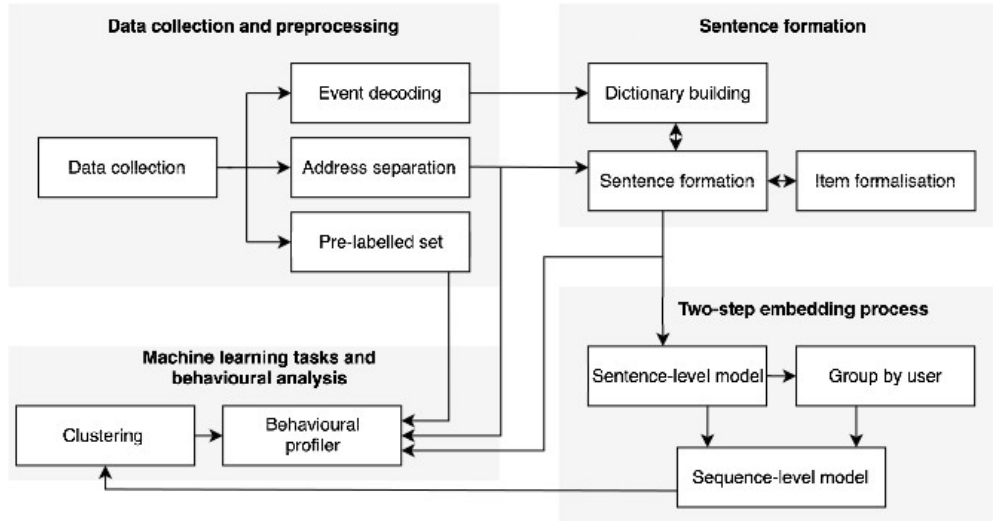


Figure 1: Overview of the proposed framework.

3.1. Data Collection and Preprocessing

Ethereum transaction data covering blocks 16250000 to 16749999 was collected from the XBlock-ETH dataset Zheng et al. (2020). For each transaction, we extract the transaction hash, sender and receiver addresses, timestamp, value, gas usage, and decoded event-log information, including event emitters, event names, method IDs, and typed arguments. Event decoding is performed through Infura API queries.

Collected transactions are examined to extract candidate addresses from: (i) from/to fields, (ii) decoded log arguments with typed address values (dictionaries of the form `{type = address, value = 0x...}`), avoiding arbitrary string mining, (iii) event emitter addresses, and (iv) created-contract fields (e.g., `contractAddress`, `createdContractAddress`, and `toCreate`). Using these signals together with XBlock-ETH and the Zelic contracts dataset², addresses are separated into contract-like entities versus EOA candidates, and contract candidates are further probed for ERC-20/721 traits via standard `eth_call` selectors, including `name`, `symbol`, `decimals`, `totalSupply`, `supportsInterface` for ERC-721 and metadata IDs, and `balanceOf (0x0)`. Some EOAs are promoted to potential users without further checks, including observed transaction senders and creator addresses labelled as EOAs in XBlock-ETH. Remaining EOA candidates are filtered by deterministic static heuristics for zero, dead, precompile, `0xeeee...`, near-zero-prefix, and small-numeric addresses, for example addresses below 2^{96} . For the remaining unresolved addresses, we query `eth_getTransactionCount` at the latest available state: addresses with `nonce  $\geq 1$` are added to the user set, while addresses with `nonce 0` are retained in the filtered set. The first iteration on the first-month window yielded 6 045 497 users, 3 117 292 contracts, 1 343 354 ERC-20 tokens, 259 062 ERC-721 tokens, with 1 317 947 addresses filtered out.

Token-level context is added where available. ERC-20 information comes from CoinGecko and ERC-721 information from OpenSea, providing metadata such as supply, price, volume, ownership, sales, and floor price. Addresses are mapped to readable user and contract IDs. Transactions are then linked to users: sent transactions are *primary*, while event-linked transactions are *secondary*. The resulting histories, address sets, and market data form the input to sentence formation.

External suspiciousness labels are collected from community and research sources, including the De.Fi REKT database³, DefiLlama⁴, DeFiHackLabs⁵, ImmuneBytes reports⁶, and the bot-address dataset of Niedermayer et al. (2024). The final pre-labelled set contains 358 unique addresses (some resources present overlaps), with category-level assignments including 68 phishing users, 50 phishing contracts, 94 malicious contracts, 96 exploiters, and 58 exploited

²<https://huggingface.co/datasets/Zellic/all-ethereum-contracts>

³<https://de.fi/rekt-database/>

⁴<https://defillama.com/hacks>

⁵<https://github.com/SunWeb3Sec/DeFiHackLabs/tree/main>

⁶<https://tinyurl.com/68796ant>

Table 2

Typical samples of final behavioural sentences.

Example	Final behavioural sentence
No decoded events	0 46.27k primary 1671841535
Primary event-rich transaction	0 104.43k primary 1671830579 transfer1 ERC20 UNI-V2 0 no market value C5352 repeats 2 sync1 C5352 swap1 C5352
Secondary event-rich transaction	0.045 332.06k secondary 1671918791 transfer1 C1013179 A320147 approvalforall1 C1013179 C368034 transfer1 C1013179 A1305885 repeats 2 transfer1 C1013179 C708 A1305885 transfer1 C1013179 A1305885 self withdrawall1 C1013179

contracts. These labels are not used to train, guide, or constrain clustering. They are introduced only after clustering and profiling as high-confidence external evidence.

3.2. Sentence Formation

Sentence formation converts transactions and decoded logs into compact behavioural text while preserving action order, event semantics, entity types, and market signals. Table 2 shows examples of final behavioural sentences. Let,

$$\mathbf{T} = \{\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_k\}$$

be the transaction set. Each transaction $\mathbf{t}_i$ contains an ordered decoded-event list

$$\Sigma(\mathbf{t}_i) = [\sigma_{i,1}, \sigma_{i,2}, \dots, \sigma_{i,l_i}],$$

where $\sigma_{i,j}$, with $1 \leq j \leq l_i$, represents a decoded event object, and l_i is the number of decoded events in $\mathbf{t}_i$.

An event dictionary maps canonical event signatures to stable aliases. For example, `Transfer(address, address, uint256)` may be mapped to `transfer1`. Let $\phi(\sigma_{i,j})$ denote the canonical signature of event $\sigma_{i,j}$, and let $\psi(\phi(\sigma_{i,j}))$ denote its alias.

For each transaction, we first construct an intermediate sentence $\tilde{s}_i$ from transaction-level metadata and ordered event-level records. Each event record contains the event alias and the addresses appearing as event emitters or typed address arguments. These addresses are grouped using the address classification from the preprocessing stage: ERC-20 tokens, ERC-721 tokens, contracts, filtered addresses, and users.

The final sentence s_i is obtained by applying the transformation $\tau(\tilde{s}_i) = s_i$. This transformation normalises transaction-level attributes, including value, gas usage, transaction type, and timestamp, into compact tokens. It also represents events by aliases, compresses repeated events with the same structure, includes token and market information on the first token occurrence within a transaction, and replaces contract and user mentions with compact identifiers. Applying τ to all intermediate sentences gives the sentence set

$$\{s_1, s_2, \dots, s_k\}.$$

Let $\mathbf{U}$ denote the set of users. For each user $\mathbf{u}_r \in \mathbf{U}$, let

$$\mathcal{Q}(\mathbf{u}_r) = [\mathbf{t}_{r,1}, \mathbf{t}_{r,2}, \dots, \mathbf{t}_{r,t_r}]$$

be the time-ordered transaction sequence linked to that user. The corresponding behavioural sentence sequence is

$$\mathcal{T}(\mathbf{u}_r) = [s_{r,1}, s_{r,2}, \dots, s_{r,t_r}],$$

where $s_{r,j}$ is derived from transaction $\mathbf{t}_{r,j}$. This sequence is the input to the two-step embedding process. Let,

$$\mathbf{U}^+ = \{\mathbf{u}_r \in \mathbf{U} \mid |\mathcal{T}(\mathbf{u}_r)| \geq \ell_{\min}\}$$

denote the users, who are retained after applying the minimum behavioural-history threshold $\ell_{\min}$.

3.3. Two-Step Embedding Process

The two-step embedding process maps each retained user's behavioural sentence sequence to a compact user-level representation. Rather than representing transactions independently, the framework embeds behavioural sentences and aggregates their ordered sequence to capture user-level routines.

In the first step, a pretrained sentence model maps each sentence to a fixed-length vector. Let

$$g_\alpha : \mathcal{D}_{\text{sent}} \rightarrow \mathbb{R}^{d_s}$$

be the sentence embedding function for model α , where $\mathcal{D}_{\text{sent}}$ is the sentence domain and d_s is the embedding dimension. The sentence embedding sequence for user $\mathbf{u}_r$ is

$$\mathbf{Z}^{\text{sent}}(\mathbf{u}_r) = [\mathbf{z}_{r,1}^{\text{sent}}, \mathbf{z}_{r,2}^{\text{sent}}, \dots, \mathbf{z}_{r,t_r}^{\text{sent}}] = [g_\alpha(s_{r,1}), g_\alpha(s_{r,2}), \dots, g_\alpha(s_{r,t_r})].$$

In the second step, the sentence embedding sequence is converted into a single user embedding. Since user histories may have different lengths, sequences longer than the chosen maximum length L are cropped according to the selected policy, while shorter sequences are padded and masked during batching. Let

$$\hat{\mathbf{Z}}^{\text{sent}}(\mathbf{u}_r) = [\hat{\mathbf{z}}_{r,1}^{\text{sent}}, \hat{\mathbf{z}}_{r,2}^{\text{sent}}, \dots, \hat{\mathbf{z}}_{r,\hat{t}_r}^{\text{sent}}]$$

denote the processed sequence, where $\hat{t}_r \leq \min(t_r, L)$. A sequence aggregation model then maps this sequence to a user-level vector. Let

$$h_\beta : (\mathbb{R}^{d_s})^{\hat{t}_r} \rightarrow \mathbb{R}^{d_s}$$

be the aggregation function for sequence model β . The final embedding of user $\mathbf{u}_r$ is

$$\mathbf{z}_r = h_\beta(\hat{\mathbf{Z}}^{\text{sent}}(\mathbf{u}_r)).$$

Applying this process to all retained users yields

$$\mathcal{E} = \{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_m\},$$

where $m = |\mathbf{U}^+|$. Each vector represents one retained user's behavioural history and is used for downstream clustering.

3.4. Clustering

Given the user embedding set $\mathcal{E}$, the clustering stage assigns retained users to behavioural groups. Formally, the objective is to learn a mapping

$$f : \mathbf{U}^+ \rightarrow \{C_1, C_2, \dots, C_n\}.$$

where $C_1, \dots, C_n$ denote the discovered behavioural clusters. Users with similar learned behavioural representations are clustered together, revealing recurring retained-user patterns. The stage is fully unsupervised: pre-labelled addresses and external references are not used to train or constrain the clustering; they are incorporated only afterwards during profiling.

3.5. Behavioural Profiler

After clustering, the framework applies a behavioural profiler to describe the discovered groups. The profiler combines cluster membership with behavioural artefacts produced by the earlier stages, including intermediate and final behavioural sentences and the associated contract and token interactions. From these inputs, it derives motif abstractions, ordered motif sequences, activity and temporal profiles, entity-exposure profiles, and contract and token diversity or concentration patterns. This allows each cluster to be interpreted not only as a geometric grouping in the embedding space, but also as a behavioural group with shared actions, routines, and interaction patterns.

Available risk evidence, including pre-labelled addresses, risky entity references, and exploit-transaction references, is introduced only during profiling. It is used to interpret and evaluate groups after clustering rather than to learn user representations or determine cluster membership.

Behavioural base. The behavioural base brings together the information needed to describe each user's activity. For a user $\mathbf{u}_r \in \mathbf{U}$, the profiler collects the corresponding intermediate and final behavioural sentence sequences, together with the contracts and tokens involved in those behaviours. We represent this user-level information as

$$p_r = (\mathbf{u}_r, \tilde{\mathcal{T}}(\mathbf{u}_r), \mathcal{T}(\mathbf{u}_r), \mathbf{R}_r, \mathbf{N}_r),$$

where $\tilde{\mathcal{T}}(\mathbf{u}_r)$ and $\mathcal{T}(\mathbf{u}_r)$ denote the intermediate and final behavioural sentence sequences, and $\mathbf{R}_r \subseteq \mathbf{R}$ and $\mathbf{N}_r \subseteq \mathbf{N}$ denote the associated contracts and tokens.

Although the implementation organises the underlying information as transaction-level records, p_r denotes the combined user-level profiler record for $\mathbf{u}_r$. Throughout this sub-section, the subscript r therefore refers to the same user and to the profiling artefacts derived for that user.

The complete profiler set is

$$\mathcal{D}_{\text{prof}} = \{p_r \mid \mathbf{u}_r \in \mathbf{U}\}.$$

After applying the global minimum behavioural-history threshold, the retained profiler set is

$$\mathcal{D}_{\text{prof}}^+ = \{p_r \mid \mathbf{u}_r \in \mathbf{U}^+\}.$$

Thus, $\mathcal{D}_{\text{prof}}^+$ contains the profiler records of the users retained for subsequent behavioural analysis.

Ordered motif artefacts. For each retained user, the profiler extracts motifs from the behavioural sentence sequence. A motif is a compact behavioural unit preserving the main action and important entities involved, such as a token transfer, contract-mediated swap, approval, NFT transfer, or claim event. Let

$$\mu : \mathcal{D}_{\text{sent}} \rightarrow \mathcal{M}^*$$

denote the motif extraction function, where $\mathcal{M}^*$ is the space of finite motif sequences. The ordered motif sequence of $\mathbf{u}_r$ is

$$\mathbf{M}(\mathbf{u}_r) = \mu(s_{r,1}) \parallel \mu(s_{r,2}) \parallel \dots \parallel \mu(s_{r,t_r}),$$

where $\parallel$ denotes sequence concatenation. Its compact summary is

$$\mathbf{B}(\mathbf{u}_r) = \text{Summary}(\mathbf{M}(\mathbf{u}_r)).$$

The sequence $\mathbf{M}(\mathbf{u}_r)$ preserves behavioural order, while $\mathbf{B}(\mathbf{u}_r)$ provides a compact user-level summary for cluster-level comparison.

Cluster membership. The profiler links the retained behavioural records to the clustering result f . The retained users assigned to cluster C_j are

$$\mathbf{U}_j = \{\mathbf{u}_r \in \mathbf{U}^+ \mid f(\mathbf{u}_r) = C_j\}.$$

User-level evidence. For each retained user $\mathbf{u}_r$, the profiler constructs complementary activity, entity-exposure, and direct risk-label profiles.

The activity profile of $\mathbf{u}_r$ is

$$\pi_r^{\text{act}} \equiv \pi^{\text{act}}(\mathbf{u}_r) = \text{ActProf}(p_r, \mathbf{M}(\mathbf{u}_r)).$$

The function ActProf derives activity measurements from the sentence histories in p_r and the ordered motif sequence $\mathbf{M}(\mathbf{u}_r)$. The resulting profile records activity volume, the number of represented transactions, first and last activity times, active span, transaction frequency, inter-record gaps, and coarse temporal patterns.

The entity-exposure profile is

$$\pi_r^{\text{ent}} \equiv \pi^{\text{ent}}(\mathbf{u}_r) = \text{EntProf}(p_r, \mathcal{L}_{\text{ent}}),$$

where $\mathcal{L}_{\text{ent}}$ denotes the available risky entity references.

The function EntProf analyses the contract and token interactions represented in p_r . It records the distinct entities involved, counts repeated interactions, measures entity diversity and concentration, identifies frequently interacted entities, and compares them with the risky entity references in $\mathcal{L}_{\text{ent}}$.

The direct risk-label profile is

$$\pi_r^{\text{lab}} \equiv \pi^{\text{lab}}(\mathbf{u}_r) = \text{LabProf}(\mathbf{u}_r, \mathcal{L}_{\text{addr}}),$$

where $\mathcal{L}_{\text{addr}}$ is the pre-labelled address set.

The function LabProf retrieves the entries in $\mathcal{L}_{\text{addr}}$ whose address is $\mathbf{u}_r$ and records the associated risk labels, if any. These labels are attached only after user representations and cluster memberships have been produced.

Cluster-level evidence. The profiler aggregates the retained user-level evidence within each cluster.

The cluster risk profile and its risk-type breakdown are obtained by aggregating the direct risk-label and entity-exposure profiles of the cluster members:

$$(\Pi_j^{\text{risk}}, \Delta_j^{\text{risk}}) = \text{RiskProf}(\{(\pi^{\text{lab}}(\mathbf{u}_r), \pi^{\text{ent}}(\mathbf{u}_r)) \mid \mathbf{u}_r \in \mathbf{U}_j\}).$$

For each cluster member, RiskProf combines the user's direct risk-label and entity-exposure profiles and aggregates this evidence across $\mathbf{U}_j$. The resulting profile Π_j^{risk} summarises the available cluster-level risk evidence, while Δ_j^{risk} provides its breakdown by risk type.

The exploit-transaction profile is

$$\Pi_j^{\text{tx}} = \text{TxProf}(\mathbf{U}_j, \{p_r \mid \mathbf{u}_r \in \mathbf{U}_j\}, \mathcal{L}_{\text{tx}}),$$

where $\mathcal{L}_{\text{tx}}$ denotes the available exploit-transaction references.

The sentence histories stored in p_r preserve the source-transaction references associated with each sentence, allowing the represented behaviour to be traced back to the transactions from which it was derived. The function TxProf matches these transaction references against $\mathcal{L}_{\text{tx}}$ and aggregates the resulting matches. The profile Π_j^{tx} records the number and proportion of users with exploit-transaction exposure, the proportion of represented activity involving such transactions, and the concentration of the exposure across cluster members. Contact with an exploit-labelled transaction is treated as contextual evidence and does not by itself imply that the entire cluster is malicious.

The defining motifs of cluster C_j are obtained by comparing the motif summaries of users inside and outside the cluster:

$$D_j^{\text{mot}} = \text{MotifRank}(\{B(\mathbf{u}_r) \mid \mathbf{u}_r \in \mathbf{U}_j\}, \{B(\mathbf{u}_r) \mid \mathbf{u}_r \in \mathbf{U}^+ \setminus \mathbf{U}_j\}).$$

Similarly, the defining motif sequences are

$$D_j^{\text{seq}} = \text{SeqRank}(\{M(\mathbf{u}_r) \mid \mathbf{u}_r \in \mathbf{U}_j\}, \{M(\mathbf{u}_r) \mid \mathbf{u}_r \in \mathbf{U}^+ \setminus \mathbf{U}_j\}).$$

The functions MotifRank and SeqRank compare the evidence observed among users inside C_j with that observed among retained users outside the cluster. They rank motifs and motif sequences using their occurrence frequency, user coverage, and cluster-versus-rest distinctiveness. The resulting artefacts therefore describe both which behavioural units distinguish a cluster and how those units are ordered. Their semantic interpretation is

$$\omega_j = \text{Read}(D_j^{\text{mot}}, D_j^{\text{seq}}).$$

The function Read converts the ranked motif and sequence evidence into a human-understandable description of the cluster's principal actions, interacted entity types, and recurring ordered routines.

The cluster temporal and diversity profile is

$$\Pi_j^{\text{td}} = \text{TDProf}(\{p_r \mid \mathbf{u}_r \in \mathbf{U}_j\}).$$

The function TDProf derives temporal and entity-diversity statistics directly from the profiler records of the users in C_j . The resulting profile Π_j^{td} summarises temporal activity patterns, activity distributions, contract and token diversity, and entity concentration.

Multi-label category assignment. Let $\mathcal{G}_{\text{tag}}$ denote the set of behavioural and risk-oriented tags used by the profiler. For each cluster C_j , the profiler assigns tags using the user-level evidence of its members and the evidence aggregated for the cluster:

$$\kappa_j = \text{CatProf} \left(\mathbf{U}_j, \left\{ (\pi^{\text{act}}(\mathbf{u}_r), \pi^{\text{ent}}(\mathbf{u}_r), \pi^{\text{lab}}(\mathbf{u}_r)) \mid \mathbf{u}_r \in \mathbf{U}_j \right\}, \Pi_j^{\text{risk}}, \Delta_j^{\text{risk}}, \Pi_j^{\text{tx}}, D_j^{\text{mot}}, D_j^{\text{seq}}, \omega_j, \Pi_j^{\text{td}} \right).$$

The function CatProf compares the available user-level and cluster-level evidence with the behavioural and risk-oriented indicators associated with each candidate tag $g \in \mathcal{G}_{\text{tag}}$. A tag is assigned when the available evidence provides sufficient support for the corresponding interpretation. The resulting assignment records the strength of the evidence and the behavioural or risk-oriented observations that justify the tag.

The output κ_j summarises the behavioural and risk-oriented tags supported for C_j , together with the strength, supporting evidence, and justification for each assignment:

$$\kappa_j = \left\{ (g, \sigma_j(g), E_j(g), J_j(g)) \mid g \in \mathcal{G}_{\text{tag}} \text{ is supported by the available evidence} \right\}.$$

For each assigned tag, $\sigma_j(g) \in [0, 1]$ records the strength of the supporting evidence, $E_j(g)$ identifies the observations supporting the assignment, and $J_j(g)$ provides a concise justification. The origin of the evidence is retained for traceability. Because κ_j is multi-label, a cluster may receive several behavioural or risk-oriented tags.

The final profile of cluster C_j is

$$F_j = \text{RenderProf}(\mathbf{U}_j, \kappa_j, \omega_j, D_j^{\text{mot}}, D_j^{\text{seq}}, \Pi_j^{\text{td}}).$$

The function RenderProf organises the cluster membership, assigned tags, semantic reading, defining motifs and sequences, and temporal and diversity profiles into an interpretable cluster report F_j . Behavioural and risk-oriented supporting evidence is carried through κ_j , including its evidence, assignment reasons, and provenance.

For each tag g , let

$$C_g = \{C_j \mid g \text{ is assigned to } C_j\}$$

denote the clusters carrying that tag. The tag-level summary is

$$\Psi(g) = \text{TagProf}(\{F_j \mid C_j \in C_g\}).$$

The function TagProf collects the final profiles of all clusters carrying tag g and aggregates their recurring motifs, sequences, temporal patterns, entity patterns, and supporting risk evidence into the tag-level summary $\Psi(g)$. Because a cluster can receive multiple tags, the same cluster profile may contribute to several tag-level summaries.

For risk-oriented tags, $\sigma_j(g)$ reflects how strongly the available evidence supports a suspicious interpretation, it does not represent the probability that users in the cluster are malicious. Evidence limited to interactions with risky entities or exploit-related transactions is treated as contextual exposure rather than sufficient grounds to characterise the entire cluster as malicious.

4. Experiments

This section addresses RQ1 and RQ2 by selecting the most suitable framework setting and comparing it with baseline and alternative methods. It compares sentence embeddings, sequence representations, clustering configurations, runtime cost, and pre-labelled address evidence.

4.1. Runtime Information and Experimental Setup

Table 3 reports representative runtimes for the main stages of the framework. The dominant costs come from API-dependent preprocessing, especially event decoding and address separation, followed by sentence embedding. In contrast, sentence construction, sequence-model training, sequence embedding generation, and clustering are comparatively lightweight once address artefacts and sentence embeddings have been produced. This supports the practicality of the framework for million-user-scale behavioural analysis, particularly when existing data can be reused across runs.

Table 3

Typical runtimes for core components of the framework.

Stage	Runtime
Event decoding	9 hours
Address separation	10 hours
ERC-20 enrichment	few minutes
ERC-721 enrichment	few minutes
Readable ID assignment	1 minute
Build intermediate sentences	20 minutes
Group intermediate sentences	30 mininutes
Build final sentences	1 hour
Group final sentences	few minutes
Sentence embedding	11 hours, 2 models
Sequence-model training	~1 minute/model
Sequence embedding generation	~1 minute/model
Clustering	few minutes
Overall core pipeline	~32 hours

Table 4Sentence embedding model selection and slice-based sanity check. *Lower global similarity mean and higher global similarity standard deviation indicate a less-collapsed embedding space.*

(a) Best configuration per sentence embedding model										
Model	Setting	Prompt	Norm	Score	r@10	AUC _{act}	AUC ₁	Margin	Global $\mu \pm \sigma$	Sent/s
MiniLM-L6-v2	num_bucket	none	post_norm	3.354	0.724	0.712	0.982	0.445	0.575 ± 0.201	2786
BGE-base	num_bucket	retrieval	encode_norm	3.265	0.708	0.720	0.991	0.290	0.708 ± 0.123	726
BGE-small	num_bucket	retrieval	post_norm	3.241	0.713	0.745	0.989	0.224	0.796 ± 0.104	1278
GTE-small	num_bucket	none	encode_norm	3.170	0.688	0.756	0.992	0.106	0.903 ± 0.046	1272
E5-small	num_bucket	none	raw_dot	3.117	0.689	0.728	0.979	0.089	0.910 ± 0.042	1277

(b) Slice-based sanity check using post_norm										
Model	Slice A					Slice B				
	Global $\mu \pm \sigma$	Margin	r@10	AUC _{act}	Sent/s	Global $\mu \pm \sigma$	Margin	r@10	AUC _{act}	Sent/s
MiniLM-L6-v2	0.570 ± 0.197	0.452	0.723	0.683	1724	0.585 ± 0.194	0.437	0.708	0.706	1773
BGE-base	0.704 ± 0.119	0.291	0.703	0.721	633	0.712 ± 0.118	0.284	0.691	0.739	632
BGE-small	0.794 ± 0.102	0.224	0.715	0.730	1008	0.801 ± 0.100	0.218	0.701	0.759	1001

Experimental Setup Implementation was developed in Python with PyTorch. Experiments were run on a workstation with a 13th Gen Intel Core i9-13900 CPU, 128GB RAM, and an NVIDIA GeForce RTX 4090 GPU with 24GB memory.

4.2. RQ1: Which Framework Setting is the Most Suitable for Behavioural Analysis?

This sub-section evaluates the two-step embedding design space on one month of data, covering blocks 16250000 to 16499999. It compares sentence embedding and preprocessing choices, sequence aggregation methods, clustering algorithms and resolutions. The strongest candidates are then assessed through behavioural semantics analysis, since a suitable configuration should recover stable clusters while preserving meaningful behavioural patterns.

4.2.1. Analysis of Embedding Choices at Sentence Level

Blockchain transaction sentences are highly templated, so embedding collapse is a key risk: distinct behaviours may be mapped into a narrow similarity band. We therefore evaluated sentence embedding models using weak-label diagnostics that measure both semantic discrimination and geometric spread, including local ranking quality $r@10$, alias-level separation AUC₁, activity-level separation AUC_{act}, separation margin, and global similarity mean and standard deviation. These diagnostics assess representation quality rather than malicious/benign classification. Positive

Table 5

Sequence aggregation model selection and sanity-audit results. *Step 1* scores are reported as $\text{mean} \pm \text{std}$ across $k \in \{15, 20, 25, 50, 100\}$ and 5 seeds. Lower median row time indicates higher throughput. Lower η^2 gap indicates less sequence-length leakage.

(a) Top Step 1 configurations ranked by aggregated clustering score										
Seq. model	Sent. model	Norm	Step-1 score	Row time (s)	Tier					
CNN	BGE-base	raw	0.962 ± 0.035	91.3	A					
GRU	BGE-base	raw	0.853 ± 0.015	94.4	A					
GRU	BGE-base	L2	0.838 ± 0.007	97.7	B					
CNN	MiniLM-L6-v2	L2	0.818 ± 0.037	42.6	B					
Mamba2	BGE-small	L2	0.803 ± 0.028	43.7	B					
CNN	MiniLM-L6-v2	raw	0.786 ± 0.036	42.7	C					
Mean	BGE-base	raw	0.782 ± 0.024	91.8	C					
GRU	MiniLM-L6-v2	raw	0.731 ± 0.037	42.8	C					
Mean	BGE-base	L2	0.740 ± 0.012	92.1	C					
GRU	BGE-small	raw	0.712 ± 0.071	42.3	C					

(b) Combined Step 1 robustness and Step 2 sanity summary										
Seq. model	Sent. model	Norm	Step-1	T1	Sanity	T2	η^2_{len}	η^2 gap	Overall	Tier
Mamba2	BGE-small	L2	0.803 ± 0.028	B	0.530 ± 0.194	C	0.344	0.344	0.817	A
CNN	MiniLM-L6-v2	L2	0.818 ± 0.037	B	0.316 ± 0.048	C	0.603	0.603	0.793	A
GRU	MiniLM-L6-v2	raw	0.731 ± 0.037	C	0.527 ± 0.164	C	0.309	0.309	0.751	B
Mean	MiniLM-L6-v2	L2	0.749 ± 0.010	D	0.388 ± 0.027	C	0.336	0.336	0.614	C
GRU	BGE-small	raw	0.712 ± 0.071	C	0.340 ± 0.171	D	0.300	0.300	0.610	C
GRU	MiniLM-L6-v2	L2	0.680 ± 0.003	D	0.492 ± 0.076	B	0.277	0.277	0.596	C
GRU	BGE-small	L2	0.621 ± 0.052	E	0.460 ± 0.074	B	0.308	0.308	0.483	D
Mamba2	MiniLM-L6-v2	L2	0.680 ± 0.020	D	0.331 ± 0.102	D	0.328	0.328	0.365	D

pairs are constructed from shared action aliases, token signatures, or inferred activity categories such as swap, borrow, transfer, and approval, excluding broad “other” cases from activity-level AUC computation.

We compared MiniLM-L6-v2 (Wang et al., 2020; Reimers and Gurevych, 2019), BGE-base and BGE-small (Xiao et al., 2023), GTE-small (Li et al., 2023), and E5-small (Wang et al., 2022) across six preprocessing settings, six prompt variants, and three normalisation modes, using up to 50 000 users, 12 000 sentences, at most six sentences per user, batch size 1024, and bf16 precision. As shown in Table 4, MiniLM-L6-v2 with num_bucket preprocessing, no prompt, and post_norm achieved the best overall profile, with the lowest global similarity mean, highest global similarity spread, and largest separation margin, while maintaining strong $r@10$ and AUC values. The slice-based sanity check in Table 4(b) supports this ranking: MiniLM-L6-v2 preserved its expanded-geometry pattern across slices, while BGE-base and BGE-small were stable but more compressed. We therefore use MiniLM-L6-v2 with num_bucket preprocessing and no prompt as the primary sentence embedding configuration, retaining BGE-small and BGE-base with num_bucket and the retrieval prompt as robustness alternatives.

4.2.2. Analysis of Embedding Choices at Sequence Level

After selecting sentence encoders, we evaluated how each user’s sentence sequence should be aggregated into a single behavioural vector. The selection process used two checks: a multi- k , multi-seed clustering audit to measure cluster quality, stability, cross- k smoothness, balance, and runtime; and a leakage audit to test whether clusters were mainly driven by trivial signals such as sequence length or activity volume.

Step 1: Multi- k Clustering Audit We evaluated mean pooling, GRU (Cho et al., 2014), TX (Vaswani et al., 2017), CNN (Kim, 2014), and Mamba2 (Dao and Gu, 2024) over MiniLM-L6-v2, BGE-small, and BGE-base sentence embeddings, using both raw and L2-normalised user vectors. MiniBatch K -means was run with $k \in \{15, 20, 25, 50, 100\}$ and five random seeds, measuring clustering quality, seed stability, cross- k alignment, centroid drift, cluster balance, and speed. Table 5(a) shows that CNN and GRU with BGE-base achieved the strongest Step 1 scores, with CNN $\times$ BGE-base $\times$ raw ranked first. However, BGE-base was substantially slower than MiniLM-L6-v2

Table 6

Training-objective and sequence-length sweeps for selected learned sequence models. *Panel (a) reports the top two objectives for each selected learned setting with $\max_len = 64$, with ranges over $k \in \{15, 20, 25, 50, 100\}$. Panel (b) reports the best length and cropping configuration for each selected target. Lower $\text{act_}\eta^2$ indicates less activity-volume confounding.*

(a) Training objective sweep							
Sentence model	Seq. model	Norm	Objective	SC range	stabARI range	DBI	maxFrac
MiniLM-L6-v2	GRU	L2	masked	0.454–0.480	0.771–0.922	1.354	0.298
MiniLM-L6-v2	GRU	L2	nextstep	0.413–0.452	0.849–0.962	1.507	0.331
MiniLM-L6-v2	GRU	raw	nextstep	0.400–0.429	0.832–0.959	1.568	0.322
MiniLM-L6-v2	GRU	raw	masked	0.189–0.458	0.841–0.927	1.403	0.315
BGE-small	GRU	L2	masked	0.462–0.494	0.854–0.969	1.349	0.307
BGE-small	GRU	L2	nextstep	0.436–0.463	0.864–0.964	1.455	0.312
BGE-small	GRU	raw	masked	0.445–0.518	0.822–0.972	1.305	0.302
BGE-small	GRU	raw	nextstep	0.443–0.484	0.838–0.933	1.418	0.288
MiniLM-L6-v2	Mamba2	L2	nextstep	0.409–0.429	0.806–0.961	1.665	0.319
MiniLM-L6-v2	Mamba2	L2	masked	0.406–0.434	0.866–0.957	1.914	0.324

(b) Length and cropping sweep						
Target	Max len.	Sampling setting	Score	ARI	$\text{act_}\eta^2$	
GRU masked + MiniLM-L6-v2 (L2)	32	prefix, sqrt_len	0.301 ± 0.026	0.921	0.000	85
GRU nextstep + MiniLM-L6-v2 (raw)	128	prefix, uniform	0.241 ± 0.015	0.890	0.000	61
GRU masked + BGE-small (L2)	128	stratified_window, sqrt_len	0.319 ± 0.020	0.917	0.000	98
GRU masked + BGE-small (raw)	32	stratified_window, sqrt_len	0.315 ± 0.031	0.892	0.001	03
Mamba2 nextstep + MiniLM-L6-v2 (L2)	32	prefix, uniform	0.218 ± 0.018	0.879	0.000	69

and BGE-small. Among the efficient configurations, $\text{CNN} \times \text{MiniLM-L6-v2} \times \text{L2}$, $\text{Mamba2} \times \text{BGE-small} \times \text{L2}$, and GRU variants with MiniLM-L6-v2 or BGE-small remained competitive.

Step 2: Sanity Audit Since strong clustering scores can reflect trivial activity signals, we tested whether clusters were mainly driven by sequence length or activity volume. The sanity audit focused on the more cost-efficient sentence models, MiniLM-L6-v2 and BGE-small, and measured length-to-cluster association using η^2 , mutual information, within-bin dominance, effective cluster count, and permuted-label baselines. Table 5(b) shows that CNN retained strong Step 1 performance but also had substantial length leakage, especially $\text{CNN} \times \text{MiniLM-L6-v2} \times \text{L2}$ with an η^2 gap of 0.603. GRU variants had the lowest leakage, with η^2 gaps around 0.277 to 0.309, while Mean and Mamba2 were intermediate. We therefore carried forward GRU configurations with MiniLM-L6-v2 and BGE-small under both raw and L2 normalisation, kept $\text{Mamba2} \times \text{MiniLM-L6-v2} \times \text{L2}$ as a complementary model, and retained $\text{Mean} \times \text{MiniLM-L6-v2} \times \text{L2}$ as a simple baseline. CNN was excluded from the behavioural analysis because of length leakage, and BGE-base was not prioritised due to higher runtime and lack of sanity-audit coverage.

Learned Sequence Models: Training Objective Sweep and Length Optimisation After selecting the main sequence aggregators, we trained compact GRU and Mamba2 models to capture behavioural dynamics from sentence embeddings. The sweep covered $\text{GRU} \times \text{MiniLM-L6-v2}$ with raw/L2 normalisation, $\text{GRU} \times \text{BGE-small}$ with raw/L2 normalisation, and $\text{Mamba2} \times \text{MiniLM-L6-v2}$ with L2 normalisation. Training used $\max_len = 64$, $\text{batch_users_train} = 256$, 2000 GPU steps with AMP, and MiniBatch K -means evaluation over $k \in \{15, 20, 25, 50, 100\}$ with five seeds. Table 6(a) shows that the best objective was setting-dependent: masked was strongest for most GRU configurations, while $\text{GRU} \times \text{MiniLM-L6-v2}$ with raw normalisation and $\text{Mamba2} \times \text{MiniLM-L6-v2}$ with L2 favoured nextstep.

We also swept sequence length, cropping, and sampling settings to justify bounded histories. In the first-month data, 6 045 497 users had median sequence length 2, $p_{90} = 9$, and $p_{99} = 76$, but a long tail up to 7.46×10^5 events. Table 6(b) shows that the best setting was target-dependent: GRU variants preferred either $\max_len = 32$ or 128, with prefix or stratified_window cropping depending on the sentence model and normalisation, while Mamba2

Table 7

k -sweep peaks, stability, and elbow-search caps. Metrics are reported as mean $\pm$ std over $R = 5$ runs on X_{eval} . k^* denotes the best k from the sweep, and k_{max} denotes the maximum k retained for the subsequent elbow search.

Configuration	k^*	k_{max}	SC	DBI	NMI / AMI
Mean $\times$ MiniLM-L6-v2 (L2)	10	30	0.4385 ± 0.0129	1.8210 ± 0.0493	0.7932/0.7931
GRU $\times$ MiniLM-L6-v2 (masked, L2)	10	60	0.4881 ± 0.0131	1.2516 ± 0.0636	0.8456/0.8456
GRU $\times$ MiniLM-L6-v2 (nextstep, raw)	60	80	0.4372 ± 0.0045	1.5896 ± 0.0466	0.8230/0.8225
GRU $\times$ BGE-small (masked, L2)	20	60	0.4813 ± 0.0120	1.3537 ± 0.0370	0.8321/0.8321
GRU $\times$ BGE-small (masked, raw)	100	140	0.5104 ± 0.0098	1.3888 ± 0.0172	0.8535/0.8525
Mamba2 $\times$ MiniLM-L6-v2 (nextstep, L2)	10	60	0.4387 ± 0.0340	1.4681 ± 0.0522	0.7952/0.7952

performed best with $\text{max_len} = 32$, prefix cropping, and uniform sampling. These capped settings cover most users while limiting compute and reducing domination by extreme-length accounts.

Final Configurations Based on Structural Evidence Based on the objective and length sweeps, we proceeded with six configurations: (i) a non-learned baseline using mean pooling over MiniLM-L6-v2 embeddings with L2 normalisation; (ii) GRU $\times$ MiniLM-L6-v2 $\times$ masked $\times$ L2 with $\text{max_len}=32$ (prefix, sqrt_len); (iii) GRU $\times$ MiniLM-L6-v2 $\times$ nextstep $\times$ raw with $\text{max_len}=128$ (prefix, uniform); (iv) GRU $\times$ BGE-small $\times$ masked $\times$ L2 with $\text{max_len}=128$ (stratified_window, sqrt_len); (v) GRU $\times$ BGE-small $\times$ masked $\times$ raw with $\text{max_len}=32$ (stratified_window, sqrt_len); and (vi) Mamba2 $\times$ MiniLM-L6-v2 $\times$ nextstep $\times$ L2 with $\text{max_len}=32$ (prefix, uniform).

4.2.3. Analysis of Clustering Configurations

To select the clustering configuration for downstream behavioural analysis, we first ran a k -sweep consistency study for each embedding configuration and then compared several clustering algorithms at the selected resolutions. The comparison included full K -means (MacQueen, 1967), MiniBatch K -means (Sculley, 2010), FAISS K -means (FAISS, 2025), Leiden community detection (Traag et al., 2019), and hybrid K -means+Leiden refinement. Because many users had only one transaction, the strongest K -means settings were also re-tested after excluding length-1 users.

k -sweep Consistency Study Per configuration, we swept $k \in \{10, 20, 30, 40, 60, 80, 100, 140, 200, 300, 450, 650\}$. Each setting was evaluated over $R = 5$ runs using an evaluation set X_{eval} of 200 000 users and a fresh training set X_{train} of the same size per run. MiniBatch K -means was fitted on X_{train} , labels were predicted on X_{eval} , and clustering quality and stability metrics were recorded. Run-to-run consistency was measured using pairwise NMI and AMI.

Table 7 reports the best k for each configuration, selected by mean SC, together with DBI, NMI/AMI stability, and the k_{max} cap used for the subsequent elbow search. The results indicate that the preferred clustering resolution is configuration-dependent rather than governed by a single universal k .

Clustering Algorithm Selection After fixing a reasonable clustering resolution for each representation using the k -sweep, we compared practical clustering algorithms at the selected resolutions. To ensure comparability, clustering used cosine distance after PCA reduction to 128 dimensions. The PCA transform was trained on 400 000 sampled users, and clustering metrics were computed on a consistent evaluation sample of 200 000 users. Each final k was selected by an inertia-based elbow search capped by the preceding k -sweep.

Table 8(a) summarises the best retained clustering method for each candidate embedding configuration. Across the selected embedding spaces, the K -means family produced the most reliable geometric structure: Leiden-only clustering fragmented the space, and hybrid K -means+Leiden refinement did not improve the trade-off. Full K -means was retained for the learned GRU and Mamba2 embeddings, while refined MiniBatch K -means was retained for the mean baseline. The strongest geometric results came from masked GRU embeddings: GRU $\times$ MiniLM-L6-v2 $\times$ L2 achieved the highest SC, while GRU $\times$ BGE-small $\times$ L2 achieved the lowest DBI and highest CHI. The raw GRU variants were weaker and more length-confounded, whereas Mamba2 was retained as a complementary option because it showed lower length leakage.

A focused follow-up excluded users with only one linked transaction, reducing the population from 6 045 497 to 3 032 235. As shown in Table 8(b), the masked GRU configurations remained strongest on SC, DBI, and CHI, while

Table 8

Clustering method selection and selected configuration summary. *Panel (a) reports the best retained clustering method for each candidate embedding configuration. Panel (b) reports the selected configurations before and after excluding length-1 users. Higher SC and CHI are better; lower DBI, maximum cluster fraction, Gini, length leakage, and number of clusters below 1% are preferred. "All" denotes all users, while ≥ 2 excludes users with only one linked transaction. The column $< 1\%$ gives the number of clusters containing fewer than 1% of users.*

(a) Best clustering method per candidate embedding configuration											
Embedding	Configuration	Selected method		SC	DBI	CHI					
Mean	MiniLM-L6-v2 (L2)	MiniBatch K -means, refined		0.437	1.87	7747					
GRU	MiniLM-L6-v2, masked, L2	Full K -means, cosine		0.468	1.411	21 260					
GRU	MiniLM-L6-v2, nextstep, raw	MiniBatch K -means, refined		0.371	1.651	7206					
GRU	BGE-small, masked, L2	Full K -means, cosine		0.465	1.309	26 559					
GRU	BGE-small, masked, raw	Full K -means, cosine		0.427	1.373	11 689					
Mamba2	MiniLM-L6-v2, nextstep, L2	Full K -means, cosine		0.414	1.65	11 043					

(b) Selected clustering configurations											
Model	Setting	Users	k	SC	DBI	CHI	Max	Min	Gini	η_{len}^2	$< 1\%$
Mean	MiniLM-L6-v2 (L2)	All	20	0.437	1.872	7747.3	0.362	0.0098	0.532	0.315	2
Mean	MiniLM-L6-v2 (L2)	≥ 2	22	0.329	2.132	4382.1	0.165	0.0202	0.324	0.196	0
GRU	MiniLM-L6-v2, masked, L2	All	26	0.468	1.411	21 260.3	0.355	0.0065	0.513	0.313	2
GRU	MiniLM-L6-v2, masked, L2	≥ 2	19	0.438	1.481	12 405.3	0.176	0.0188	0.331	0.175	0
GRU	BGE-small, masked, L2	All	25	0.465	1.309	26 558.9	0.358	0.0040	0.576	0.316	4
GRU	BGE-small, masked, L2	≥ 2	26	0.420	1.524	11 579.0	0.183	0.0109	0.395	0.200	0
Mamba2	MiniLM-L6-v2, nextstep, L2	All	29	0.414	1.648	11 043.1	0.380	0.0053	0.553	0.288	5
Mamba2	MiniLM-L6-v2, nextstep, L2	≥ 2	22	0.336	1.695	8544.8	0.143	0.0137	0.315	0.171	0

the filtered partitions became more balanced and less length-driven, with no clusters below 1% of users. The carried-forward configurations use $k = 20$ for the mean baseline, $k = 26$ for GRU MiniLM, $k = 25$ for GRU BGE-small, and $k = 29$ for Mamba2 in the full-population setting; after filtering, the corresponding resolutions are $k = 22$, $k = 19$, $k = 26$, and $k = 22$. These selected configurations form the input to the behavioural semantic analysis stage.

4.2.4. Behavioural Semantics Analysis

The previous tests identified structurally strong clustering candidates, but the final setting should also preserve behavioural semantics. We therefore compared the selected configurations by measuring how well they concentrate pre-labelled phishing, other malicious, and bot addresses within individual clusters. The same 175 labelled addresses were used throughout: the full-population setting clustered 6 045 497 users and recovered 117 labelled addresses, while the length ≥ 2 setting clustered 3 032 235 users and recovered 108 labelled addresses.

Table 9 compares the strongest observed label concentrations produced by each configuration. These values are used as descriptive selection signals rather than as performance metrics. In the full-population setting, GRU $\times$ BGE-small $\times$ masked $\times$ L2 gave the strongest phishing concentration, while Mamba2 gave the strongest bot concentration. In the length ≥ 2 setting, Mamba2 achieved the strongest phishing concentration, with 15/32 phishing addresses (46.9%) in one cluster, while GRU $\times$ BGE-small $\times$ masked $\times$ L2 gave the strongest bot concentration. The other-malicious sample is small, with only 9 matched addresses, so those results are interpreted cautiously. Combining these concentration results with the structural results in Table 8(b), we select the length ≥ 2 Mamba2 $\times$ MiniLM-L6-v2 $\times$ nextstep $\times$ L2 setting for downstream behavioural analysis. It uses max_len = 32, prefix cropping, uniform user sampling, PCA-128, cosine full K -means, and $k = 22$; future runs can reselect k using the elbow method with $k_{max} = 30$.

Answer to RQ1

The length ≥ 2 Mamba2 $\times$ MiniLM-L6-v2 $\times$ nextstep $\times$ L2 setting is selected for behavioural analysis, using max_len = 32, prefix cropping, uniform sampling, PCA-128, cosine full K -means, and $k = 22$. Although the

Table 9

Pre-labelled behavioural concentration across the selected clustering configurations. “Other mal.” combines labels such as bug exploit, front run, and oracle manipulation. For phishing, other malicious, and bots, the table reports the largest count observed in any single cluster. “All” denotes all users, while ≥ 2 excludes users with only one linked transaction.

Configuration	Users	Labelled found	Phishing	Other mal.	Bots	Bot spread
Mean $\times$ MiniLM-L6-v2 (L2)	All	117/175	10/38	3/9	20/70	12
Mean $\times$ MiniLM-L6-v2 (L2)	≥ 2	108/175	10/32	2/9	19/67	15
GRU $\times$ MiniLM-L6-v2 $\times$ masked $\times$ L2	All	117/175	13/38	3/9	21/70	18
GRU $\times$ MiniLM-L6-v2 $\times$ masked $\times$ L2	≥ 2	108/175	14/32	2/9	18/67	15
GRU $\times$ BGE-small $\times$ masked $\times$ L2	All	117/175	22/38	2/9	20/70	13
GRU $\times$ BGE-small $\times$ masked $\times$ L2	≥ 2	108/175	12/32	2/9	20/67	17
Mamba2 $\times$ MiniLM-L6-v2 $\times$ nextstep $\times$ L2	All	117/175	10/38	2/9	22/70	21
Mamba2 $\times$ MiniLM-L6-v2 $\times$ nextstep $\times$ L2	≥ 2	108/175	15/32	2/9	18/67	17

Table 10

Comparison of our framework with baseline and alternative methods. SC, DBI, and CHI measure geometric clustering quality. η_c^2 , NMI_c, and Gap measure length/activity leakage. SepScore, Sep-NMI, Purity, OneClus, Distinct, and Coll. measure pre-labelled category separation.

Method	k	SC	DBI	CHI	η_c^2	NMI _c	Gap	SepScore	Sep-NMI	Purity	OneClus	Distinct	Coll.
Feature	9	0.412	1.190	3965.2	0.701	0.335	0.517	0.571	0.147	0.700	0.280	3.0	0.0
BUBA	6	0.691	0.610	60110.3	0.537	0.268	0.402	0.341	0.084	0.650	0.590	1.0	2.0
TF-IDF+SVD	10	0.192	2.420	597.4	0.299	0.151	0.223	0.593	0.199	0.752	0.324	3.0	0.0
Doc2Vec	9	0.100	2.891	445.1	0.291	0.156	0.223	0.621	0.237	0.770	0.274	3.0	0.0
Our framework	8	0.328	1.409	3048.6	0.248	0.119	0.182	0.603	0.240	0.790	0.260	2.6	0.4

masked GRU variants have stronger geometric scores, Mamba2 gives the best filtered phishing concentration and the cleanest partition, with lower cluster imbalance and weaker length leakage.

4.3. RQ2: How Does the Proposed Framework Perform Compared with Baseline and Alternative Methods?

We compare the selected framework configuration with two baselines and two alternative embedding methods. For the feature baseline, we reuse the account-level features of Valadares et al. (2023), namely transactions sent/received, transactions to/from smart contracts, Ether balance, and total transactions, and add API-derived features for transaction frequency, transaction volume, and token diversity. The second baseline is BUBA, a graph-based behavioural pipeline from our previous work (Zelenyanszki et al., 2026). As alternative embedding methods, TF-IDF+SVD and Doc2Vec replace the two-step embedding process by producing text-based user representations from the users’ sentences before clustering. Each representation is clustered with K -means using the elbow-selected k , and results are averaged over five random seeds. For TF-IDF+SVD and Doc2Vec, we use the same settings as the framework, namely (max_len=32), prefix cropping, and uniform sampling, ensuring a fair comparison. The comparison set contains 10 100 addresses from the first-month time window, consisting of pre-labelled addresses together with 10 000 additional users, where each selected address has at least two behavioural sentences and at least one transaction with events, as BUBA operates on event-bearing transactions.

Table 10 summarises the comparison across conventional clustering quality, length/activity leakage, and pre-labelled category separation. The results should therefore be read across the three metric groups rather than as a single-score ranking. BUBA achieves the strongest geometric clustering results, with the highest SC and CHI and the lowest DBI, confirming its ability to form compact graph-based user groups. Its leakage values indicate that activity scale also contributes to the recovered structure, which is expected in DeFi user-flow data where interaction intensity and behavioural structure are closely related. The feature baseline shows even stronger leakage, indicating that its clusters are heavily influenced by account-level activity statistics. TF-IDF+SVD and Doc2Vec provide useful text-based alternatives, as both separate the pre-labelled categories into distinct best clusters; however, their low SC and high DBI indicate weaker overall cluster structure. In contrast, the proposed framework does not maximise the

Table 11

First-month behavioural families.

Behaviour family	Users	Clusters
Stablecoin / transfer / approval	1 036 073	1, 2, 6, 13, 14, 18
DEX / AMM interaction	808 653	3, 5, 16, 21
NFT flow	527 614	0, 4, 8, 10, 11, 20
Mint / claim / reward	459 932	7, 12, 15, 17
ENS / name-service / bridge-like	199 963	9, 19

Table 12

First-month suspiciousness interpretation levels.

Suspiciousness level	Users	Clusters
Meaningfully suspicious	952 868	5, 8, 13, 16, 3
Watchlist	401 962	0, 9, 18, 19, 21
Mostly ordinary / exposure evidence	1 677 405	1, 2, 4, 6, 7, 10, 11, 12, 14, 15, 17, 20

geometric clustering metrics, but it achieves the lowest length/activity leakage, the highest Sep-NMI, the highest labelled-cluster purity, and the smallest one-cluster malicious concentration. This makes it the only method that remains consistently strong across cluster geometry, leakage control, and pre-labelled category separation, rather than improving one dimension at the expense of another.

Answer to RQ2

The proposed framework provides the most suitable representation for behavioural user analysis. Although BUBA achieves stronger geometric scores, the proposed framework better balances cluster quality, leakage control, and separation.

5. Behavioural Analysis

This section addresses RQ3 and RQ4 by analysing the behaviours recovered from the selected clusters and their persistence over time. It first examines the first-month clusters to identify the main behavioural families, suspiciousness levels, and behavioural indicators. It then studies monthly and cumulative windows to assess whether ordinary and suspicious behavioural tags recur beyond a single observation period.

5.1. RQ3: What Behavioural Characteristics Can Be Learned from This Type of Analysis?

We interpret clusters using the behavioural profiler in Subsection 3.5. The profiler summarises motifs, motif sequences, entity usage, temporal patterns, diversity, concentration, risk-label evidence, and risky-entity or exploit-transaction exposure. These summaries turn cluster memberships into behavioural indicators, explaining the roles, routines, and risk-associated patterns captured by the learned representations.

5.1.1. Learned Behavioural Characteristics from the First-month Analysis

The first-month clusters form a mixed behavioural landscape rather than being dominated by a single activity type. As summarised in Table 11, the main families include transfer and approval behaviour, DEX/AMM interaction, NFT flows, mint/claim/reward activity, and ENS/name-service or bridge-like activity. These families describe behavioural settings rather than threat categories: the largest families involve ordinary stablecoin, transfer, approval, DEX/AMM, NFT, and reward-related behaviours, while suspicious evidence appears as smaller pockets within them. The behavioural reading combines motifs, ordered routines, temporal activity, diversity, concentration, direct labels, and risky-entity or exploit-transaction exposure into indicators that distinguish burst-like, persistent, broad, and entity-centred behaviour.

Stablecoin, Transfer, and Approval Behaviour Stablecoin and transfer-heavy clusters form the main ordinary baseline. Clusters 2 and 14 are the clearest stablecoin-transfer groups, while Clusters 1, 6, and 18 mix transfer behaviour

with approval or exposure evidence. Cluster 13 is the main exception, combining approval management, longer activity histories, and the clearest phishing-focused signal in this family. The main indicators are repeated ERC20 transfers, stablecoin-specific movement, approval-linked transfer routines, and differences in activity span or token concentration.

DEX/AMM Behaviour DEX/AMM clusters capture swaps, routing, liquidity, pools, LP-token evidence, and repeated protocol interaction. Cluster 5 is the largest and strongest mixed-risk group, with bot, exploit, bug-exploit, phishing, and front-run evidence concentrated around a small set of entities. Cluster 3 shows active Uniswap-style trading with bot, exploit, and oracle manipulation pockets. Cluster 16 represents a broader DEX/AMM community, while Cluster 21 is best read as a DEX/approval watchlist community. These indicators distinguish ordinary trading communities from concentrated DEX groups where bot, exploit, front-run, or oracle manipulation evidence becomes behaviourally meaningful.

NFT Transfer and Approval Behaviour NFT clusters are characterised by ERC721/ERC1155 transfers, approval-for-all events, marketplace activity, minting, and repeated NFT-contract interaction. Cluster 8 is the clearest suspicious NFT cluster, where phishing and exploit evidence aligns with longer-lived NFT transfer and approval behaviour. Clusters 4, 10, and 20 provide weaker-risk NFT baselines, while Clusters 0 and 11 are better read as mixed or watchlist NFT/token-transfer communities. The key indicators are NFT transfer motifs, approval-for-all behaviour, marketplace-style interaction, minting, and repeated NFT-contract use.

Mint, Claim, and Reward Behaviour Mint, claim, and reward clusters describe campaign-like participation through claim, mint, reward, airdrop, and distribution motifs. Cluster 15 is the clearest burst-like campaign, while Clusters 12 and 17 show more persistent reward-style participation and Cluster 7 sits between burst-like and repeated reward behaviour. Risk exposure appears in this family, but the dominant behaviour remains campaign, claim, or reward participation. The key indicators are claim, mint, reward, airdrop, and distribution motifs, especially when they appear in repeated routines or are followed by token transfer.

ENS/name-service and Bridge-like Behaviour ENS/name-service and bridge-like clusters are smaller but distinct. Cluster 9 combines name-service management with NFT transfer and approval behaviour, while Cluster 19 combines name-service behaviour with bridge-like or deposit-like movement. Both are best read as specialised watchlist communities where risk evidence is embedded within coherent ENS/name-service or bridge-like behaviour. The indicators include name-service management, resolver or ownership-style updates, bridge-like or deposit-like movement, and repeated interaction with specialised identity or cross-chain entities.

The suspiciousness reading is grouped into three interpretation levels, as shown in Table 12. These are not binary normal/malicious labels, but indicate how strongly risk evidence shapes a cluster. Meaningfully suspicious clusters contain user-label or exposure evidence aligned with coherent behavioural indicators; watchlist clusters contain genuine but weaker or subtype-specific evidence; and ordinary/exposure clusters may touch risky entities or exploit-labelled transactions without being treated as cluster-wide suspicious.

The meaningfully suspicious clusters remain grounded in recognisable behaviour. Cluster 5 is a concentrated DEX/AMM community with bot, exploit, bug-exploit, phishing, and front-run evidence. Cluster 8 links NFT transfer and approval behaviour with phishing and exploit evidence, while Cluster 13 is the clearest phishing-focused case, combining approval management, phishing evidence, and asset-movement indicators. Clusters 3 and 16 are broader DEX/AMM communities where bot, exploit, oracle manipulation, exploit-transaction, and risky-entity evidence appears within trading and liquidity behaviour. In these cases, suspiciousness comes from the alignment between risk evidence and behaviour-specific indicators, not from labels alone. These examples show that the same risk label is interpreted differently depending on whether it appears in approval-heavy, NFT-centred, concentrated DEX, or broader trading behaviour.

The watchlist clusters show visible but more cautious signals. Cluster 0 combines NFT and token-transfer behaviour with sparse bug-exploit and exploit evidence; Cluster 9 links ENS/name-service activity with exploit and oracle manipulation evidence; Cluster 18 is a stablecoin-transfer watchlist cluster with bot evidence; Cluster 19 combines ENS/name-service and bridge-like behaviour with bug-exploit, exploit, exploit-transaction, and risky-entity evidence; and Cluster 21 is a DEX/approval watchlist cluster with bug-exploit, exploit, exploit-transaction, and risky-entity exposure. Ordinary/exposure clusters provide behavioural baselines: they may contact risky contracts, risky tokens,

Table 13

Suspiciousness tags and behavioural indicators.

Suspicious tag	Behavioural indicators	Reading
Phishing	Approvals, NFT movement, asset outflow	Persistent approval/NFT risk.
Bot	Repeated swaps/transfers, protocol concentration	Persistent automation.
Exploit	Risky/exploit exposure in DEX, NFT, ENS, or transfer behaviour	Risk layer in ordinary behaviour.
Bug exploit	Repeated protocol routines, exploit exposure	Persistent; clearer over time.
Front-run	DEX/AMM swaps, routing, pool interaction	Persistent DEX subtype.
Oracle manipulation	Swaps, pool/sync behaviour, liquidity use	Clearer in cumulative windows.
Rug pull	Token-centred activity, concentration, risky-token evidence	Grows from second month.
Flash-loan attack	DEX/AMM liquidity routines, exploit context	Visible from 1–3 months.
Governance attack	Governance/protocol routines, concentrated entity use	Sparse longer-window signal.
MEV exploit	DEX/AMM trading, routing, market-ordering context	Sparse longer-window subtype.
Price manipulation	DEX/AMM trading, pool/liquidity use	Only clear in 1–6 months.
Replay attack	Exploit context, repeated interaction patterns	Only clear in 1–6 months.

malicious entities, or exploit-labelled transactions, but this contact is treated as exposure rather than proof of cluster-level suspiciousness.

At the tag level, the profiler summarises recurring behavioural and risk-related evidence across clusters. Approval-linked swap and transfer tags are indicated by approvals, swaps, routing, liquidity interaction, approval-for-all events, ERC20 or NFT transfers, and outgoing asset movement. DEX trading and liquidity tags are indicated by swaps, routing, pool or sync events, LP-token evidence, and repeated protocol use. NFT market and approval tags are indicated by ERC721/ERC1155 transfers, approval-for-all, minting, listing, sale activity, and repeated NFT-contract interaction. Mint/claim tags are indicated by claim, mint, reward, airdrop, and token-distribution routines, often followed by transfer.

Other tags describe broader behavioural structure, including repeated contract or protocol interaction, stablecoin-family movement, token and contract diversity, entropy, and top-entity concentration. These indicators distinguish broad, exploratory behaviour from protocol-centred, campaign-like, or concentrated behaviour. Suspicious tags are interpreted through alignment: phishing is tied to approval management, approval-for-all, NFT movement, asset outflow, and phishing-related entity contact; bot and front-running behaviour to repeated swaps, transfers, approvals, liquidity interaction, routing, regular activity, and concentrated protocol use; and exploit, bug-exploit, and oracle manipulation signals to exploit-transaction exposure, risky-entity exposure, repeated contract interaction, and trading, NFT, transfer, or protocol-interaction settings.

Answer to RQ3

The profiler interprets clusters through behavioural families such as transfers, DEX/AMM trading, NFT activity, mint/claim behaviour, and ENS or bridge-like interaction.

Suspiciousness is treated as variation within these settings, linking phishing, bot/front-running, and exploit-related tags to coherent behavioural evidence without treating exposure as proof of cluster-wide maliciousness.

5.2. RQ4: What Are the Persistent, Long-term Behavioural Patterns?

The long-term analysis uses persistence as a validation signal. Here, *long-term* means recurring beyond one monthly window, either in the independent second month or in the cumulative 1–2, 1–3, and 1–6 month windows. The second-month window checks whether tags reappear in an independent month, while the cumulative 1–2, 1–3, and 1–6 month windows show how behavioural evidence develops over longer horizons. The analysis is tag-level rather than cluster-level because cluster identifiers are window-specific, whereas behavioural and suspiciousness tags provide a comparable vocabulary across windows. For each tag, persistence is interpreted through recurring behavioural indicators, including motifs, temporal regularity, and risky-entity exposure. Together, each recurring tag and its supporting indicators define a long-term, persistent behavioural pattern. Persistence therefore strengthens interpretation only when the recurring tag is supported by recurring behavioural evidence.

Table 14

Rarity distribution of behavioural indicators.

Rarity	Definition	Indicators
Often	Used by 5 or more tags	4
Common	Used by 3–4 tags	8
Repeated	Used by 2 tags	18
Rare	Used by 1 tag	76

Ordinary behavioural structure is stable across windows. Approval-linked transfer, approval-linked swap, DEX trading, liquidity provision, NFT market and approval behaviour, mint/claim campaigns, repeated contract or protocol interaction, and stablecoin-family movement all recur across independent and cumulative settings. Indicators include approvals, token or NFT movement, swaps, routing, pool or liquidity interaction, claim/mint routines, repeated contract touches, stablecoin token flows, and concentration or diversity patterns. Longer windows also make long-lived activity clearer, increasing from one tagged cluster in the first month to thirteen tagged clusters in the 1–6 month window. This shows that longer observation windows reveal changes in behavioural regularity and persistence, rather than just capturing more activity.

These persistent ordinary tags provide the behavioural background for suspiciousness interpretation. Approval-linked transfer and swap are common DeFi routines, but they become security-relevant when approval, transfer, asset-movement, or liquidity indicators align with phishing, exploit, bot, front-run, or oracle manipulation evidence. DEX trading is the main setting for several risk tags, while NFT market and approval behaviour is important for interpreting phishing and approval-related risk. Mint/claim campaigns and stablecoin-family movement remain mostly ordinary baselines unless stronger risk evidence aligns with their routines.

The suspiciousness tags show three broad persistence patterns. Some are visible early and persist, including phishing, bot, exploit, bug exploit, and front-run. Others become clearer in cumulative windows, especially oracle manipulation and rug pull. Sparse subtypes such as flash-loan attack, governance attack, MEV exploit, replay attack, and price manipulation mainly appear in longer windows. Table 13 summarises the behavioural indicators used to interpret these tags.

Suspicious tags are interpreted through behavioural alignment rather than labels alone. Phishing remains tied to approval management, approval-for-all, NFT movement, and asset outflow. Bot and front-running behaviour are linked to repeated DEX/AMM swaps, routing, approvals, liquidity interaction, regular activity, and concentrated protocol use. Exploit and bug-exploit evidence appears as a risk layer inside ordinary DEX, NFT, ENS/name-service, transfer, and watchlist communities, especially where risky-entity or exploit-transaction exposure aligns with repeated interaction patterns. Longer windows connect scattered actions into recognisable routines and help distinguish broad ordinary communities from narrower concentrated risk pockets.

5.2.1. Distribution of Behavioural Indicators

The indicator distribution measures how often each behavioural indicator is used across the tag vocabulary. As shown in Table 14, the distribution is strongly long-tailed: a small number of shared indicators form a DeFi and permission backbone, while most indicators remain tag-specific. Shared indicators include swaps, routing, pools, liquidity interaction, approvals, repeated protocol use, exploit or risk exposure, and concentration evidence. This suggests that the profiler is not simply detecting high activity or generic risk exposure, but separating distinct behavioural mechanisms. Approval is especially important because it connects ordinary DeFi and NFT activity with phishing, approval-heavy NFT behaviour, and exploit-related interpretations. However, approval is not suspicious by itself; it becomes meaningful only when aligned with asset movement, NFT transfer, phishing-related evidence, risky-entity exposure, or exploit-transaction exposure. Confidence therefore comes from several indicators aligning with the same behavioural reading, rather than from a single indicator.

Answer to RQ4

The long-term analysis shows persistent behavioural patterns across windows, including DEX/AMM activity, mint/claim campaigns, stablecoin movement, NFT approvals, and repeated protocol interaction. Suspiciousness

is contextual rather than categorical: phishing is linked to approvals, NFT activity, and asset movement; bot/front-running to repeated DEX activity; oracle manipulation to swap, pool, liquidity, and protocol-interaction evidence; and rug pull to token-centred concentration and campaign-like transfers. Confidence increases when multiple indicators align with direct labels, risky-entity exposure, or exploit-transaction exposure.

6. Limitations

This work has several limitations. The pre-labelled set is small because public reports cover only some attacks, bots, malicious contracts, exploiters, and exploited entities, so it is used as high-confidence validation evidence rather than exhaustive ground truth. Suspiciousness tags are cluster-level interpretive signals, not malicious/ benign labels for every user. The framework also depends on decoded events, address classification, token metadata, external risk references, and API-supported enrichment, so broader evaluation across chains, protocols, and periods is needed.

7. Conclusion and future work

This paper presented a scalable, application-agnostic framework for *persistent behavioural pattern discovery* of Ethereum users. By transforming raw transactions into behavioural sentences, learning user-level sequence representations, clustering users, and profiling the resulting groups, the framework supports population-level analysis beyond transaction-centric or attack-specific views. The results show that the discovered groups capture both ordinary behaviour, such as stablecoin transfer, DEX trading, and risk-relevant variation, including phishing, oracle manipulation, and exploit-related evidence.

The cross-window analysis shows that these patterns persist beyond a single observation period. Recurring tags reappear across independent monthly and cumulative windows, while longer windows make sparse suspicious subtypes such as rug pull, flash-loan attack, governance attack, MEV exploit, replay attack, and price manipulation easier to interpret. Future work will extend the analysis across range of chains and applications, compare with classification-based and attack-specific baselines, and evaluate the learned representations on downstream risk classification and prediction.

Data availability

The entire dataset and the code can be made available upon request.

References

- Azad, P., Akcora, C.G., Khan, A., 2025a. Machine learning for blockchain data analysis: Progress and opportunities. *Distrib. Ledger Technol.* 5. URL: <https://doi.org/10.1145/3728474>, doi:10.1145/3728474.
- Azad, P., Coskunuzer, B., Kantarcioglu, M., Akcora, C.G., 2025b. Chainlet orbits: Topological address embedding for blockchain, in: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, Association for Computing Machinery, New York, NY, USA. p. 25–36. URL: <https://doi.org/10.1145/3690624.3709322>, doi:10.1145/3690624.3709322.
- Bonifazi, G., Corradini, E., Ursino, D., Virgili, L., 2022. Defining user spectra to classify ethereum users based on their behavior. *Journal of Big Data* 9, 37.
- Cernera, F., Morgia, M.L., Mei, A., Sassi, F., 2023. Token spammers, rug pulls, and sniper bots: An analysis of the ecosystem of tokens in ethereum and in the binance smart chain (BNB), in: *32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, Anaheim, CA. pp. 3349–3366. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/cernera>.
- Cho, K., van Merriënboer, B., Gulcehre, C., et al., 2014. Learning phrase representations using rnn encoder–decoder for statistical machine translation, in: *EMNLP*.
- Dao, T., Gu, A., 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*.
- FAISS, 2025. FAISS Official Website. <https://faiss.ai/index.html>. Accessed: 2026-06-10.
- Ibba, G., Neykova, R., Ortu, M., Tonelli, R., Counsell, S., Destefanis, G., 2025. A machine learning approach to vulnerability detection combining software metrics and topic modelling: Evidence from smart contracts. *Machine Learning with Applications* 22, 100759. URL: <https://www.sciencedirect.com/science/article/pii/S2666827025001422>, doi:<https://doi.org/10.1016/j.mlwa.2025.100759>.
- Kim, Y., 2014. Convolutional neural networks for sentence classification, in: *EMNLP*.
- Li, X., et al., 2023. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*.
- Lin, D., Ding, Y., Zou, W., Chen, J., Luo, X., Wu, J., Zheng, Z., 2025a. Risktagger: An llm-based agent for automatic annotation of web3 crypto money laundering behaviors. URL: <https://arxiv.org/abs/2510.17848>, arXiv:2510.17848.

- Lin, D., Zheng, Z., Wu, J., Yang, J., Lin, K., Xiao, H., Song, B., Zheng, Z., 2025b. Track and trace: Automatically uncovering cross-chain transactions in the multi-blockchain ecosystems. *IEEE Transactions on Services Computing* 18, 4291–4303. doi:10.1109/TSC.2025.3618729.
- Ma, J., Jiang, M., Jiang, J., Luo, X., Hu, Y., Zhou, Y., Wang, Q., Zhang, F., 2025. Understanding security issues in the dao governance process. *IEEE Transactions on Software Engineering* 51, 1188–1204. doi:10.1109/TSE.2025.3543280.
- MacQueen, J., 1967. Some methods for classification and analysis of multivariate observations, in: *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, Oakland, CA, USA. pp. 281–297.
- Niedermayer, T., Saggese, P., Haslhofer, B., 2024. Detecting financial bots on the ethereum blockchain, in: *Companion Proceedings of the ACM Web Conference 2024*, p. 1742–1751.
- Reimers, N., Gurevych, I., 2019. Sentence-bert: Sentence embeddings using siamese bert-networks, in: *EMNLP-IJCNLP*.
- Sculley, D., 2010. Web-scale k-means clustering, in: *Proceedings of the 19th International Conference on World Wide Web*, Association for Computing Machinery, New York, NY, USA. p. 1177–1178. URL: <https://doi.org/10.1145/1772690.1772862>, doi:10.1145/1772690.1772862.
- Sun, D., Ma, W., Nie, L., Liu, Y., 2024. Sok: Comprehensive analysis of rug pull causes, datasets, and detection tools in defi. URL: <https://arxiv.org/abs/2403.16082>, arXiv:2403.16082.
- Torres, C.F., Steichen, M., State, R., 2019. The art of the scam: Demystifying honeypots in ethereum smart contracts, in: *28th USENIX Security Symposium (USENIX Security 19)*, USENIX Association, Santa Clara, CA. pp. 1591–1607. URL: <https://www.usenix.org/conference/usenixsecurity19/presentation/ferreira>.
- Tovanich, N., Cazabet, R., 2023. Fingerprinting bitcoin entities using money flow representation learning. *Applied Network Science* 8, 63.
- Traag, V.A., Waltman, L., Van Eck, N.J., 2019. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports* 9, 5233.
- Valadares, J.A., Villela, S.M., Bernardino, H.S., Gonçalves, G.D., Vieira, A.B., 2023. Mapping user behaviors to identify professional accounts in ethereum using semi-supervised learning. *Expert Systems with Applications* 229, 120438.
- Vaswani, A., Shazeer, N., Parmar, N., et al., 2017. Attention is all you need, in: *NeurIPS*.
- Wang, D., Wu, B., Yuan, X., Wu, L., Zhou, Y., Cui, H., 2024a. Defiguard: A price manipulation detection service in defi using graph neural networks. *IEEE Transactions on Services Computing* 17, 3345–3358. doi:10.1109/TSC.2024.3489439.
- Wang, H., Hu, Y., Wu, H., Liu, D., Peng, C., Wu, Y., Fan, M., Liu, T., 2024b. Skyeye: Detecting imminent attacks via analyzing adversarial smart contracts, in: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, Association for Computing Machinery, New York, NY, USA. p. 1570–1582. URL: <https://doi.org/10.1145/3691620.3695526>, doi:10.1145/3691620.3695526.
- Wang, L., Yang, N., Huang, X., et al., 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*.
- Wang, W., et al., 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *NeurIPS*.
- Werner, S., Perez, D., Gudgeon, L., Klages-Mundt, A., Harz, D., Knottenbelt, W., 2023. Sok: Decentralized finance (defi), in: *Proceedings of the 4th ACM Conference on Advances in Financial Technologies*, Association for Computing Machinery, New York, NY, USA. p. 30–46. URL: <https://doi.org/10.1145/3558535.3559780>, doi:10.1145/3558535.3559780.
- Wu, C., Cao, H., Chen, J., Yan, X., Xu, G., Zhao, Z., Liu, Y., Jiang, H., 2025. Rugpull: Leveraging temporal graph neural network for rugpull detection in defi. *IEEE Transactions on Information Forensics and Security* 20, 11120–11133. doi:10.1109/TIFS.2025.3620669.
- Xiao, S., Liu, Z., et al., 2023. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*.
- Xu, X., Mao, Z., Su, J., Lin, X., Basin, D., Sun, J., Wang, J., 2025. Quantitative runtime monitoring of ethereum transaction attacks, in: *Proceedings of the ACM on Web Conference 2025*, Association for Computing Machinery, New York, NY, USA. p. 4146–4159. URL: <https://doi.org/10.1145/3696410.3714682>, doi:10.1145/3696410.3714682.
- Zelenyanszki, D., Hóu, Z., Biswas, K., Muthukkumarasamy, V., 2026. A graph neural network approach to cluster user behaviours in decentralised finance. *Applied Soft Computing*, 115102URL: <https://www.sciencedirect.com/science/article/pii/S1568494626005508>, doi:https://doi.org/10.1016/j.asoc.2026.115102.
- Zheng, P., Zheng, Z., Wu, J., Dai, H.N., 2020. Xblock-eth: Extracting and exploring blockchain data from ethereum. *IEEE Open Journal of the Computer Society* 1, 95–106. doi:10.1109/ojcs.2020.2990458.
- Zhong, J., Wu, D., Liu, Y., Xie, M., Liu, Y., Li, Y., Liu, N., 2025a. Detecting various defi price manipulations with llm reasoning. URL: <https://arxiv.org/abs/2502.11521>, arXiv:2502.11521.
- Zhong, Z., Chen, J., Wang, J., Xue, Q., Wu, J., Liu, L., Ying, X., Zheng, Z., 2025b. Towards exploring developers’ struggles in developing upgradeable smart contracts. *IEEE Transactions on Software Engineering*, 1–16doi:10.1109/TSE.2025.3609077.